Use of force-controlled grippers

1 Python control API description

Note : The time interval between calling each instruction of the gripper must be greater than 1.5 seconds.

`set_pro_gripper(gripper_id, address, value)`

- **Function** : Set the parameters of the Pro force-controlled gripper. You can set a variety of parameter functions. Please see the following table for details.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
  - `address` (`int`): The command number of the gripper.
  - `value` : the parameter value corresponding to the instruction number.

Function	gripper_id	address	value
Set the gripper ID	14	3	1 ~ 254
Set the gripper enable state	14	10	0 or 1, 0 - disable; 1 - enable
Set the clockwise runnable error of the gripper	14	21	0 ~ 16
Set the anti-clockwise runnable error of the gripper	14	23	0 ~ 16
Setting the minimum actuation force of the gripper	14	25	0 ~ 254
IO output settings	14	29	0, 1, 16, 17
Set the IO opening angle	14	30	0 ~ 100
Set IO closing angle	14	31	0 ~ 100
Set the virtual position value of the servo	14	41	0 ~ 100
Setting the holding current	14	43	1 ~ 254

- Return Value
  - Please see the following table:

Function	Return Value
Set the gripper ID	0 - failed; 1 - successful
Set the gripper enable state	0 - failed; 1 - successful

Function	Return Value
Set the clockwise runnable error of the gripper	0 - failed; 1 - successful
Set the anti-clockwise runnable error of the gripper	0 - failed; 1 - successful
Setting the minimum actuation force of the gripper	0 - failed; 1 - successful
IO output settings	0 - failed; 1 - successful
Set the IO opening angle	0 - failed; 1 - successful
Set IO closing angle	0 - failed; 1 - successful
Set the virtual position value of the servo	0 - failed; 1 - successful
Setting the holding current	0 - failed; 1 - successful

`get_pro_gripper(gripper_id, address)`

- **Function** : Get the parameters of Pro force-controlled gripper, and you can get various parameter functions. Please see the following table for details.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
  - `address` (`int`): The command number of the gripper.

Function	gripper_id	address
Read the firmware major version number	14	1
Read the firmware minor version number	14	2
Read the gripper ID	14	3
Read the clockwise runnable error of the gripper	14	22
Read the anti-clockwise runnable error of the gripper	14	24
Read the minimum actuation force of the gripper jaws	14	26
Read IO opening angle	14	34
Read IO closing angle	14	35
Get the amount of data in the current queue	14	40
Read the virtual position value of the servo	14	42
Reading the holding current	14	44

- Return value:
  - Check the following table (if the return value is -1, it means no data can be read):

Function	Return Value
Read the firmware major version number	Major version number
Read the firmware minor version number	Minor version number
Read the gripper ID	1 ~ 254
Read the clockwise runnable error of the gripper	0 ~ 254
Read the anti-clockwise runnable error of the gripper	0 ~ 254
Read the minimum actuation force of the gripper jaws	0 ~ 254
Read IO opening angle	0 ~ 100
Read IO closing angle	0 ~ 100
Get the amount of data in the current queue	Returns the amount of data in the current absolute control queue
Read the virtual position value of the servo	0 ~ 100
Reading the holding current	1 ~ 254

`set_pro_gripper_angle(gripper_id, gripper_angle)`

- **Function** : Set the angle of the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
  - `gripper_angle` (`int`): Gripper angle, value range 0 ~ 100.
- Return value:
  - 0 - Failed
  - 1 - Success

`get_pro_gripper_angle(gripper_id)`

- **Function** : Read the angle of the force-controlled gripper jaws.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- **Return value** : `int` 0 ~ 100

`set_pro_gripper_open(gripper_id)`

- **Function** : Open the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:
  - 0 - Failed
  - 1 - Success

`set_pro_gripper_close(gripper_id)`

- **Function** : Close the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:
  - 0 - Failed
  - 1 - Success

`set_pro_gripper_calibration(gripper_id)`

- **Function** : Set the zero position of the force-controlled gripper. (The zero position needs to be set first when using it for the first time)
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:
  - 0 - Failed
  - 1 - Success

`get_pro_gripper_status(gripper_id)`

- **Function** : Read the clamping status of the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:
  - `0` - Moving.
  - `1` - Motion stopped, no clamping detected.
  - `2` - Stop motion, clamping object detected.
  - `3` - After the clamping object is detected, the object falls.

`set_pro_gripper_torque(gripper_id, torque_value)`

- **Function** : Set the torque of the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.

- `torque_value` (`int`): Torque value, ranging from 1 to 100.
- Return value:
  - 0 - Failed
  - 1 - Success

`get_pro_gripper_torque(gripper_id)`

- **Function** : Read the torque of the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- **Return value:** (`int`) 100 ~ 300

`set_pro_gripper_speed(gripper_id, speed)`

- **Function** : Set the force-controlled gripper speed.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
  - `speed` (int): Gripper movement speed, ranging from 1 to 100.
- Return value:
  - 0 - Failed
  - 1 - Success

`get_pro_gripper_default_speed(gripper_id, speed)`

- **Function** : Read the default speed of the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
- **Return value** : the default movement speed of the gripper, ranging from 1 to 100.

`set_pro_gripper_abs_angle(gripper_id, gripper_angle)`

- **Function** : Set the absolute angle of the force-controlled gripper.
- parameter:
  - `gripper_id` (`int`): gripper ID, default value is 14, value range is 1 ~ 254.
  - `gripper_angle` (`int`): Gripper angle, value range 0 ~ 100.
- Return value:
  - 0 - Failed
  - 1 - Success

`set_pro_gripper_pause(gripper_id)`

- **Function** : Pause movement.
- parameter:
  - `gripper_id` (`int`) Gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:

- 0 - Failed
- 1 - Success

`set_pro_gripper_resume(gripper_id)`

- **Function** : Sports recovery.
- parameter:
  - `gripper_id` (`int`) Gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:
  - 0 - Failed
  - 1 - Success

`set_pro_gripper_stop(gripper_id)`

- **Function** : Stop motion.
- parameter:
  - `gripper_id` (`int`) Gripper ID, default value is 14, value range is 1 ~ 254.
- Return value:
  - 0 - Failed
  - 1 - Success

2 Python zero calibration instructions

After attaching the gripper to the end of the robotic arm, run the following code, and the gripper will first open to its maximum, then fully close to complete the calibration.

```
from pymycobot import MyCobot320
mc=MyCobot320("COM1")#Enter the serial port number of the robot
mc.set_pro_gripper_calibration(14)#The default ID of the gripper is 14
```

3 Python control scenario examples

demo1: torque value adaptation

The user can observe the deformation degree of the force-controlled gripper after clamping the object under the premise of setting the corresponding torque value and select a torque value with the smallest deformation but can clamp the object normally. This torque value is the most suitable torque value for the current object.

```
from pymycobot import MyCobot320
import time
mc=MyCobot320("COM1")#Fill in the serial port number of the robot
mc.set_pro_gripper_torque(14,10)#Set the torque value
print(mc.get_pro_gripper_torque(14))#View the set torque value
mc.set_pro_gripper_open(14)#Gripper fully opens
time.sleep(3)
mc.set_pro_gripper_close(14)#Gripper fully closes
```

demo2: grab and move objects at points AB

```
from pymycobot import MyCobot320
import time
mc=MyCobot320("COM1")#Fill in the serial port number of the robot
mc.set_pro_gripper_torque(14,10)#Set the torque value of the gripper
print(mc.get_pro_gripper_torque(14))#Print the set gripper torque value
mc.set_pro_gripper_open(14)#First open the gripper completely
start_angles=[19.68, -1.23, -91.4, -0.52, 90.08, 60.29]
target_coords=[[231.3, -61.3, 232.7, 178.35, -2.7, -130.56],[231.3, 65.3, 232.7, 178.35, -2.7, -130.56]]
end_angles=[80,0,-85,0,90,60]
for i in range(len(target_coords)):
    mc.sync_send_angles(start_angles,50)
    mc.send_coords(target_coords[i],100,1)
    time.sleep(2)
    mc.send_coord(3,165,50)
    time.sleep(2)
    mc.set_pro_gripper_close(14)
    time.sleep(2)
    mc.send_coords(target_coords[i],100,1)
    mc.sync_send_angles(end_angles,100)
    mc.set_pro_gripper_open(14)
    time.sleep(2)
```